

# Diabetes Risk Predictor

A PyTorch neural network and Streamlit application for clinical diabetes risk prediction

From exploratory data analysis to deployment | Technical report v1.0

**100,000**

patient records

**15**

input features

**90.6%**

test accuracy \*

**93.5%**

recall (diabetes) \*

*\* Measured on a SMOTE-balanced hold-out split; see Sections 6 and 7 for interpretation and caveats.*

PREPARED BY

Mahan Liaghatmand

DATE

September 2026

VERSION

1.0

LICENSE

MIT

Technology stack

PyTorch

scikit-learn

imbalanced-learn

pandas

Streamlit

Plotly

# Contents

- 1. Executive Summary . . . . . 3
- 2. Introduction . . . . . 3
  - 2.1 Background and motivation . . . . . 3
  - 2.2 Problem statement . . . . . 4
  - 2.3 Objectives . . . . . 4
  - 2.4 Deliverables . . . . . 4
- 3. Dataset and Exploratory Analysis . . . . . 4
  - 3.1 Dataset overview . . . . . 4
  - 3.2 Data quality . . . . . 5
  - 3.3 Class distribution . . . . . 5
  - 3.4 Feature distributions . . . . . 6
  - 3.5 Prevalence across segments . . . . . 7
  - 3.6 Correlation analysis . . . . . 9
  - 3.7 Feature importance . . . . . 9
- 4. Methodology . . . . . 10
  - 4.1 Feature encoding . . . . . 11
  - 4.2 Handling class imbalance with SMOTE . . . . . 11
  - 4.3 Train / validation / test split . . . . . 11
  - 4.4 A note on feature scaling . . . . . 11
- 5. Model Architecture and Training . . . . . 11
  - 5.1 Architecture . . . . . 11
  - 5.2 Training configuration . . . . . 12
  - 5.3 Training behaviour . . . . . 13
- 6. Evaluation and Results . . . . . 13
  - 6.1 Interpretation . . . . . 14
- 7. Critical Analysis and Supplementary Results . . . . . 15
  - 7.1 What the balanced test set hides . . . . . 15
  - 7.2 Reference baselines at the natural class balance . . . . . 15
  - 7.3 Additional observations . . . . . 16
- 8. The Streamlit Application . . . . . 16
  - 8.1 Interface . . . . . 16
  - 8.2 Inference pipeline . . . . . 17
  - 8.3 Batch file format . . . . . 17
  - 8.4 Running the project . . . . . 17
- 9. Limitations and Future Work . . . . . 18
  - 9.1 Limitations . . . . . 18
  - 9.2 Roadmap . . . . . 18
- 10. Conclusion . . . . . 19
- Appendix A. Model Input Vector . . . . . 19

# 1. Executive Summary

Diabetes is a chronic metabolic condition whose early detection allows timely lifestyle and clinical intervention. This project builds an end-to-end machine learning system that estimates a person's diabetes risk from eight routinely collected attributes: gender, age, hypertension, heart disease, smoking history, body-mass index (BMI), HbA1c level and blood glucose level. The system covers the full lifecycle: data exploration, preprocessing, class balancing, model design and training in **PyTorch**, evaluation, and delivery through an interactive **Streamlit** web application.

The model is a compact fully connected neural network (15-64-32-16-8-1, 3,777 trainable parameters) trained on 100,000 patient records, of which 8.5% are diabetic. Because of this imbalance, the minority class was oversampled with SMOTE. On the held-out test split the network reaches an accuracy of **90.56%** and a recall of **93.46%** for the diabetic class (precision 88.35%, F1-score 0.9083).

|                          |                           |                        |                          |
|--------------------------|---------------------------|------------------------|--------------------------|
| <b>90.6%</b><br>Accuracy | <b>88.4%</b><br>Precision | <b>93.5%</b><br>Recall | <b>0.908</b><br>F1-score |
|--------------------------|---------------------------|------------------------|--------------------------|

## Key findings

- **HbA1c and blood glucose dominate.** Together they account for about 80% of the feature importance measured with a Gradient Boosting model (51.8% and 28.3%); age contributes a further 8.3%.
- **The data has a sharp threshold structure.** Every patient with HbA1c of 6.8% or more, or glucose of 220 mg/dL or more, is diabetic, and nobody with HbA1c up to 5.0% or glucose up to 100 mg/dL is (Section 3.5).
- **The application is fully functional.** It provides single-patient prediction with an adjustable decision threshold, batch scoring from CSV files, and interactive data exploration.
- **Reported metrics are optimistic.** SMOTE was applied before the train/test split, and the test set is class-balanced, so the figures do not describe real-world performance. Section 7 quantifies this and Section 9 lists the corrective steps.

## Bottom line

The project delivers a complete, working prediction pipeline and a polished user interface. Its main value is as a demonstration of the machine learning workflow; before any wider use, the evaluation protocol should be rebuilt as described in the roadmap.

# 2. Introduction

## 2.1 Background and motivation

Screening tools that estimate diabetes risk from a handful of easily measured variables can help prioritise who should receive confirmatory testing. Machine learning models are attractive for this task because they learn non-linear combinations of risk factors directly from data. The project was designed as a practical learning exercise in applied machine learning: it deliberately spans the entire pipeline rather than a single notebook, so that every stage from raw CSV to a deployed interface is exercised.

## 2.2 Problem statement

Given a patient described by eight attributes, predict the probability that the patient has diabetes. Formally this is a supervised **binary classification** problem with target diabetes in {0, 1}. The model outputs a probability between 0 and 1, which is converted into a label using a configurable decision threshold (default 0.5).

## 2.3 Objectives

- Explore and document the dataset, including its quality issues and class imbalance.
- Build a reproducible preprocessing pipeline (encoding, oversampling, splitting).
- Design, train and evaluate a neural network classifier in PyTorch.
- Package the trained model and encoder as artifacts and expose them through a user-friendly Streamlit application.
- Critically assess the results and identify concrete improvements.

## 2.4 Deliverables

| Deliverable       | File                                    | Purpose                                                  |
|-------------------|-----------------------------------------|----------------------------------------------------------|
| Analysis notebook | Notebook_Project.ipynb                  | Exploration, preprocessing, training and evaluation      |
| Model weights     | artifacts/model_diabet.pt               | Trained PyTorch state_dict (19 KB)                       |
| Fitted encoder    | artifacts/OneHotEncoder.pkl             | One-hot encoder reused at inference time                 |
| Web application   | app.py                                  | Streamlit interface: prediction, batch scoring, insights |
| Dataset           | Dataset/diabetes_prediction_dataset.csv | 100,000 patient records (3.8 MB)                         |
| Documentation     | README.md, LICENSE                      | Setup instructions and MIT license                       |

Table 1. Project deliverables.

# 3. Dataset and Exploratory Analysis

## 3.1 Dataset overview

The dataset contains **100,000 records** and nine columns: eight input features and the binary target. Table 2 summarises each column. The target is diabetic for 8,500 patients (8.5%).

| Feature             | Type            | Range / categories  | Summary                    | Description                                        |
|---------------------|-----------------|---------------------|----------------------------|----------------------------------------------------|
| gender              | Categorical     | Female, Male, Other | 58,552 / 41,430 / 18       | Patient gender                                     |
| age                 | Numeric         | 0.08 to 80          | mean 41.9                  | Age in years                                       |
| hypertension        | Binary          | 0 / 1               | 7.5% positive              | History of high blood pressure                     |
| heart_disease       | Binary          | 0 / 1               | 3.9% positive              | History of heart disease                           |
| smoking_history     | Categorical     | 6 categories        | No Info 35.8%, never 35.1% | never, No Info, current, former, ever, not current |
| bmi                 | Numeric         | 10.01 to 95.69      | mean 27.32                 | Body-mass index (kg/m <sup>2</sup> )               |
| HbA1c_level         | Numeric         | 3.5 to 9.0          | mean 5.53                  | Average blood sugar over ~2-3 months (%)           |
| blood_glucose_level | Numeric         | 80 to 300           | mean 138.1                 | Blood glucose (mg/dL)                              |
| diabetes            | Binary (target) | 0 / 1               | 8.5% positive              | Diabetes diagnosis                                 |

Table 2. Feature dictionary.

3.2 Data quality

- **No missing values.** All nine columns are fully populated.
- **Duplicates.** 3,854 rows (3.85%) are exact duplicates. They were kept in the notebook; they are almost all non-diabetic (0.5% positive), so they mildly inflate the majority class.
- **Discretised laboratory values.** HbA1c takes only 18 distinct values and blood glucose only 18 distinct values across all 100,000 records, which is unusual for real measurements.
- **Repeated BMI value.** 25,495 records share the exact BMI of 27.32, the dataset mean, suggesting that missing BMIs were imputed with the average.
- **Sparse categories.** Only 18 patients have gender “Other”, and 35.8% of smoking histories are “No Info”.
- **Class imbalance.** Only 8.5% of patients are diabetic (a 10.8 : 1 ratio).

These characteristics indicate a dataset that is partly synthetic or heavily pre-processed. They do not prevent modelling, but they limit how far conclusions can be generalised to real clinical populations.

3.3 Class distribution

The left panel of Figure 1 shows the natural class split. Training directly on this distribution would favour the majority class, so a resampling strategy is needed; the right panel shows the effect of SMOTE, which is described in Section 4.

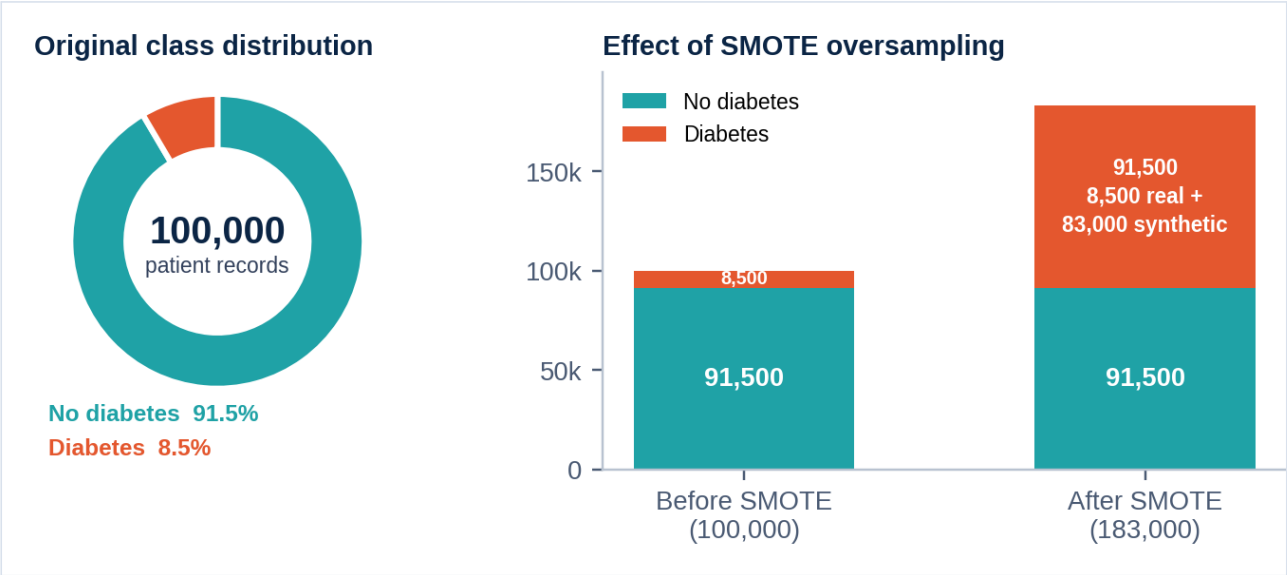


Figure 1. Class distribution before and after SMOTE oversampling.

3.4 Feature distributions

Diabetic patients are clearly older, heavier, and have higher HbA1c and glucose values (Table 3). Age and BMI overlap substantially between the classes, whereas HbA1c and glucose separate them more cleanly, which is also visible in Figure 2.

| Group mean  | Age (years) | BMI (kg/m²) | HbA1c (%) | Glucose (mg/dL) |
|-------------|-------------|-------------|-----------|-----------------|
| No diabetes | 40.12       | 26.89       | 5.40      | 132.85          |
| Diabetes    | 60.95       | 31.99       | 6.93      | 194.09          |
| Difference  | +20.83      | +5.10       | +1.54     | +61.24          |

Table 3. Mean value of the numeric features for each class.

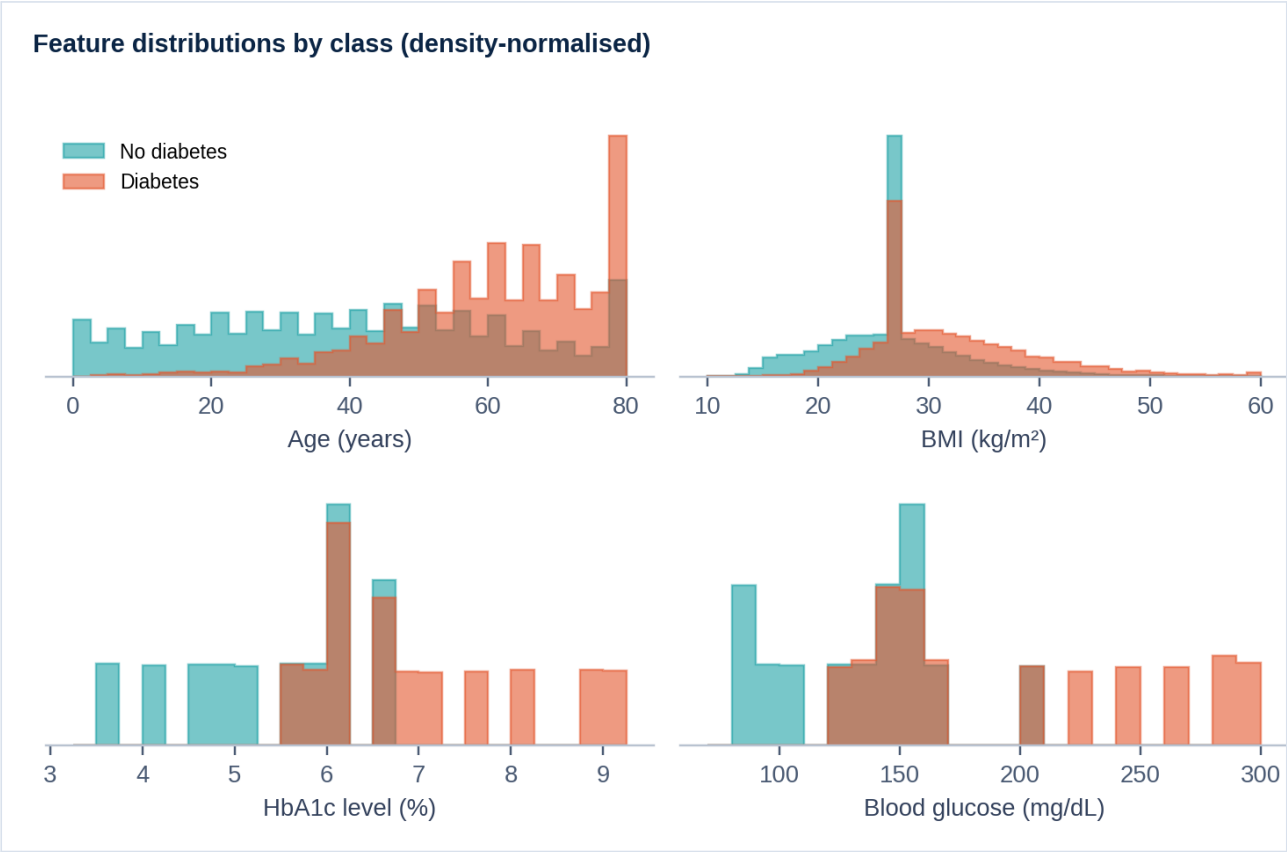


Figure 2. Density-normalised feature distributions for diabetic and non-diabetic patients. Values are clipped to the plotted range.

3.5 Prevalence across segments

Diabetes prevalence rises steadily with age (0.5% among patients up to 20 years old versus 20.0% among 61-80 year-olds) and with BMI (0.8% for under 18.5 versus 22.2% above 35). Patients with hypertension are four times as likely to be diabetic (27.9% vs 6.9%), and those with heart disease more than four times (32.1% vs 7.5%). Men show a somewhat higher rate than women (9.7% vs 7.6%). Former smokers have the highest rate among smoking categories (17.0%), while “No Info” has the lowest (4.1%).

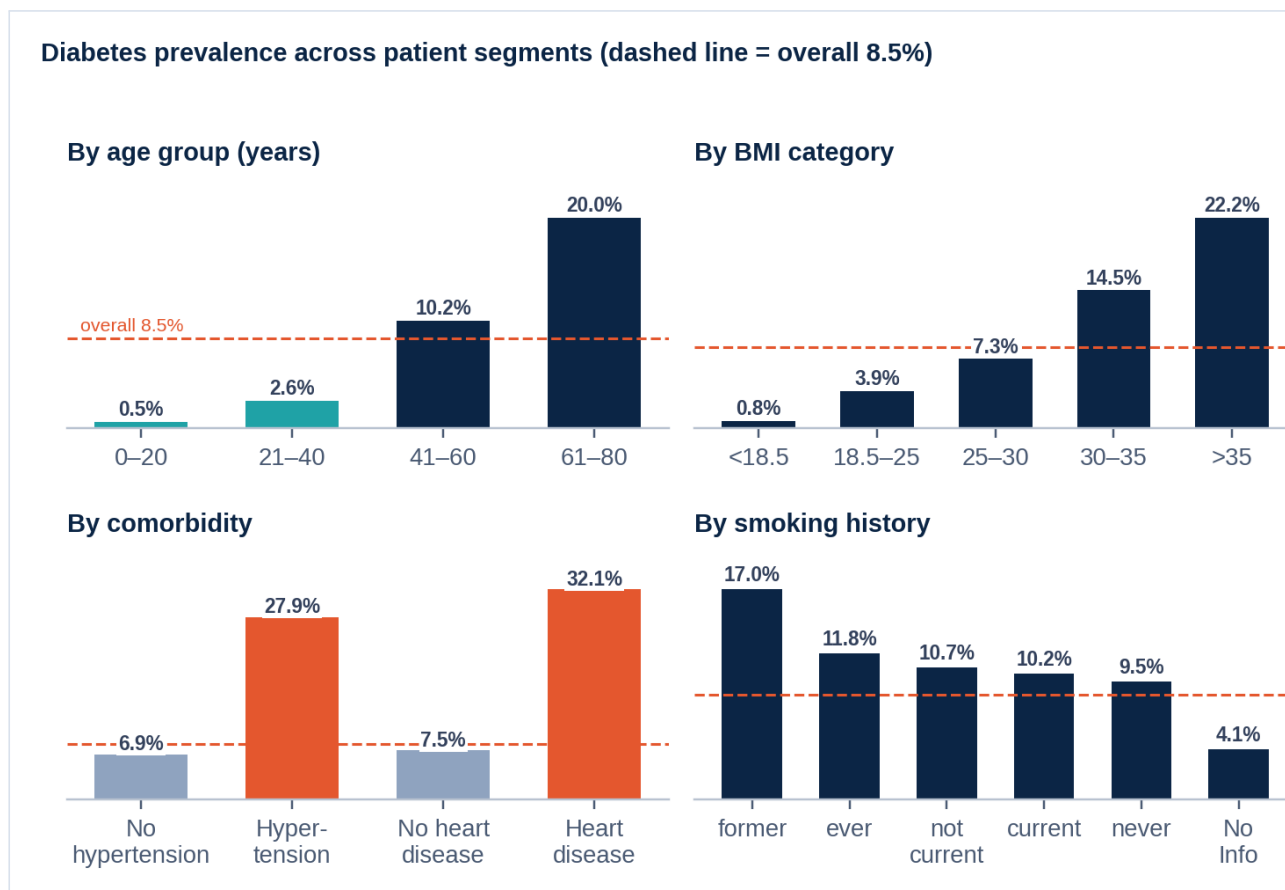


Figure 3. Diabetes prevalence by age group, BMI category, comorbidity and smoking history.

### A striking threshold structure in HbA1c and glucose

Plotting the diabetes rate at every observed HbA1c and glucose value (Figure 4) reveals a step function. Patients with HbA1c up to 5.0% or glucose up to 100 mg/dL (with a single additional glucose value of 158 mg/dL) are never diabetic; patients with HbA1c of 6.8% or more, or glucose of 220 mg/dL or more, are always diabetic. In between, the rate is roughly constant at 7-9% regardless of the exact value.

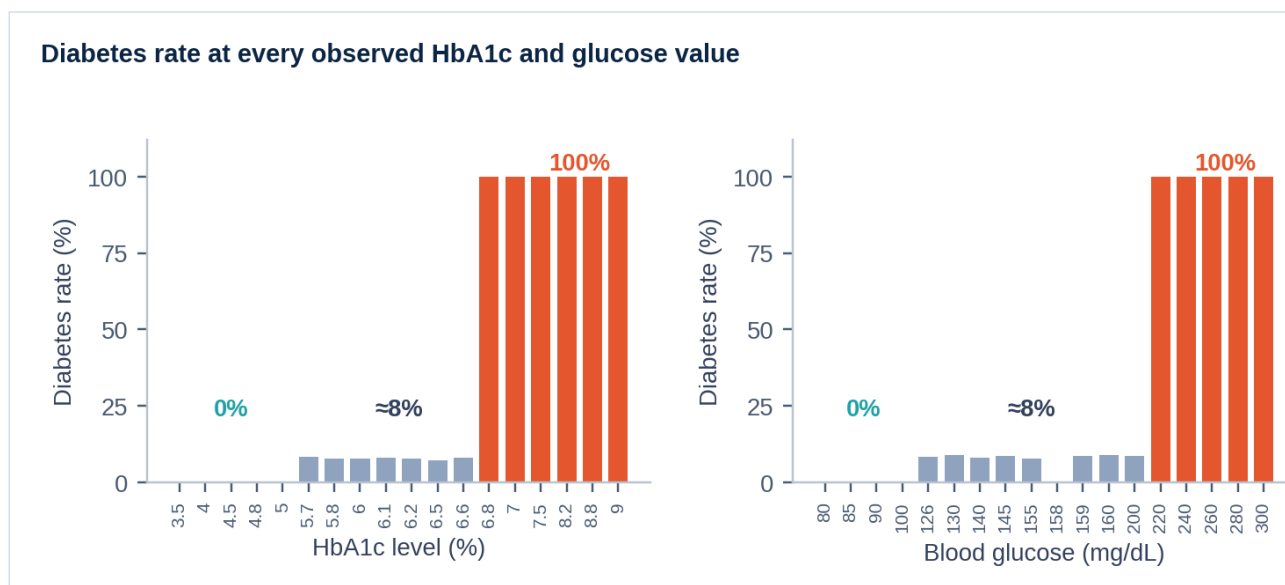


Figure 4. Diabetes rate at each distinct HbA1c and blood glucose value. Bars are coloured by rate: 0% (teal), around 8% (grey), 100% (coral).

### Why this matters

A simple two-condition rule ( $\text{HbA1c} \geq 6.8$  or  $\text{glucose} \geq 220$ ) already identifies 5,687 of the 8,500 diabetic patients (66.9%) with no false positives. The remaining 39,921 patients form an “ambiguous zone” with 7.0% prevalence, where age, BMI and comorbidities must carry the signal. Any model should be judged on how well it handles this zone.

## 3.6 Correlation analysis

Glucose ( $r = 0.42$ ) and HbA1c ( $r = 0.40$ ) have the strongest linear correlation with the target, followed by age (0.26), BMI (0.21), hypertension (0.20) and heart disease (0.17). The predictors themselves are only weakly correlated with one another (largest: age and BMI,  $r = 0.34$ ), so multicollinearity is not a concern.

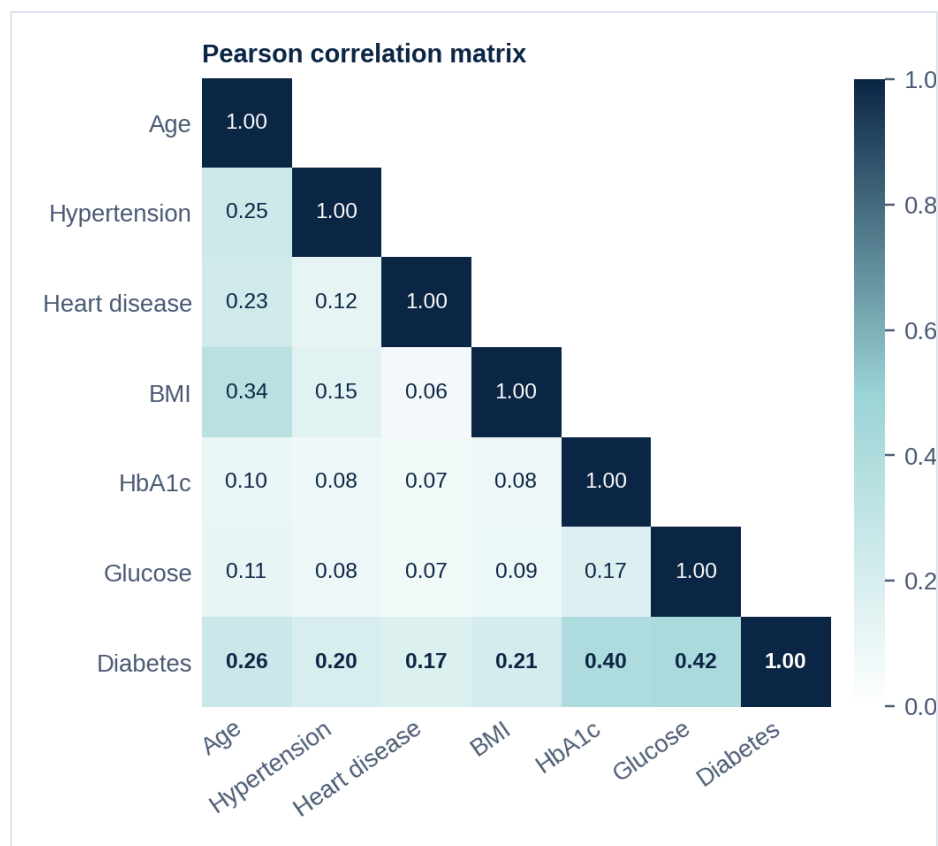


Figure 5. Pearson correlation matrix of the numeric and binary variables (lower triangle).

## 3.7 Feature importance

As a model-agnostic sanity check, a Gradient Boosting classifier was fitted in the notebook on the encoded features. It agrees with the correlation analysis: HbA1c and glucose together carry roughly 80% of the importance, and the categorical features add very little.

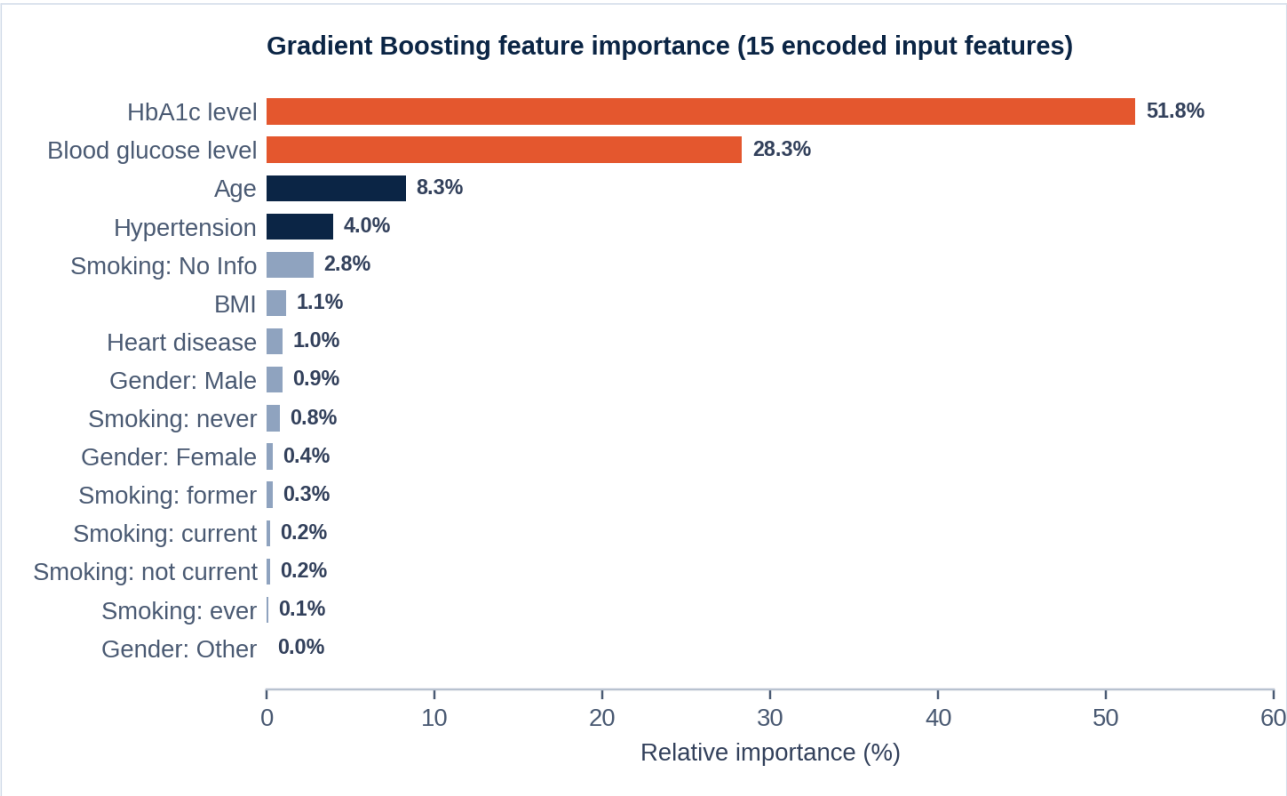


Figure 6. Relative feature importance from a Gradient Boosting classifier (values sum to 100%).

## 4. Methodology

Figure 7 gives an overview of the pipeline implemented in the notebook. Each step is explained in the sections that follow.

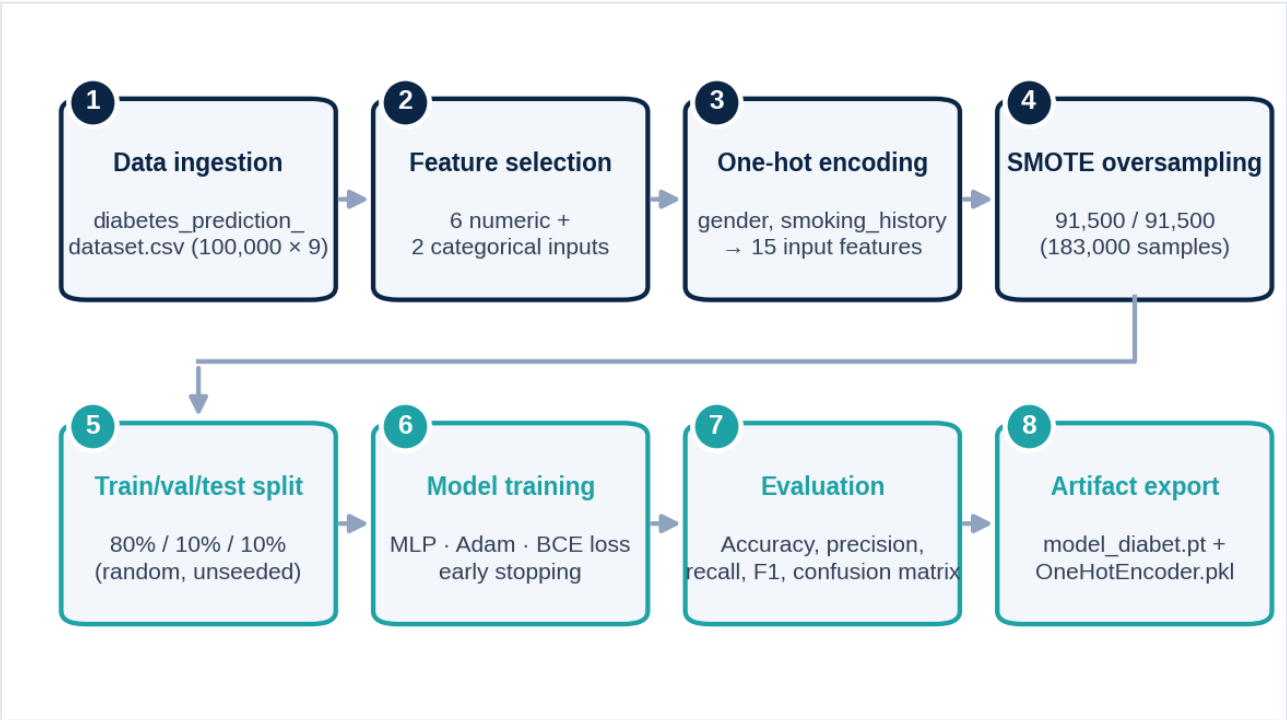


Figure 7. End-to-end machine learning pipeline, from raw data to exported artifacts.

## 4.1 Feature encoding

The six numeric and binary columns are used as they are. The two categorical columns are converted with scikit-learn's OneHotEncoder: gender yields 3 columns (Female, Male, Other) and smoking history yields 6 columns (No Info, current, ever, former, never, not current). The final input vector therefore has  $6 + 3 + 6 = 15$  **features** (Appendix A). The fitted encoder is saved to disk so that inference applies exactly the same mapping.

## 4.2 Handling class imbalance with SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) creates new minority-class examples by interpolating between a real minority sample and one of its nearest minority neighbours (Chawla et al., 2002). It was configured with `random_state=42`. It added 83,000 synthetic diabetic samples, turning the 91,500 / 8,500 split into 91,500 / 91,500 (183,000 samples in total).

## 4.3 Train / validation / test split

The balanced data was randomly divided 80% / 10% / 10% (no random seed was set, so exact numbers vary slightly between runs).

| Split      | Share | Samples | Purpose                                           |
|------------|-------|---------|---------------------------------------------------|
| Training   | 80%   | 146,400 | Weight updates                                    |
| Validation | 10%   | 18,300  | Early stopping and monitoring                     |
| Test       | 10%   | 18,300  | Final evaluation (9,139 negative, 9,161 positive) |

*Table 4. Data split after SMOTE (183,000 samples).*

## 4.4 A note on feature scaling

A `StandardScaler` was fitted in the notebook, but its output was never assigned back to the feature matrix. The network was therefore trained, and is served, on **unscaled** values. Training and inference are consistent with one another (the application deliberately omits scaling, as noted in its source code), but the choice probably slows convergence because HbA1c (about 3.5-9) and glucose (80-300) live on very different scales.

# 5. Model Architecture and Training

## 5.1 Architecture

The classifier is a multilayer perceptron with four hidden layers whose widths halve at every step (64, 32, 16, 8), a dropout layer with probability 0.5 just before the output, and a single sigmoid output that represents the probability of diabetes. The funnel shape compresses the 15 inputs into progressively more abstract representations, while the compact size (3,777 parameters) keeps the model fast enough to run in real time on a CPU.

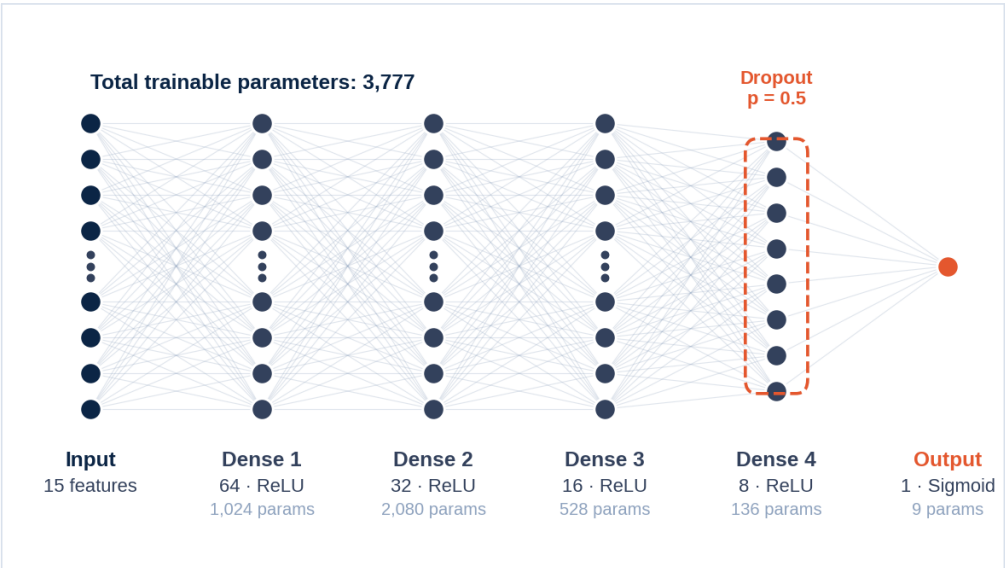


Figure 8. Network architecture 15-64-32-16-8-1. The dashed box marks the dropout layer applied to the last hidden layer during training.

| Layer | Type              | Input | Output | Activation | Parameters |
|-------|-------------------|-------|--------|------------|------------|
| 1     | Linear            | 15    | 64     | ReLU       | 1,024      |
| 2     | Linear            | 64    | 32     | ReLU       | 2,080      |
| 3     | Linear            | 32    | 16     | ReLU       | 528        |
| 4     | Linear            | 16    | 8      | ReLU       | 136        |
| -     | Dropout (p = 0.5) | 8     | 8      | -          | 0          |
| 5     | Linear            | 8     | 1      | Sigmoid    | 9          |
| Total |                   |       |        |            | 3,777      |

Table 5. Layer-by-layer summary. Parameters = inputs x outputs + biases.

5.2 Training configuration

| Setting            | Value                                                             |
|--------------------|-------------------------------------------------------------------|
| Framework          | PyTorch (CPU-compatible; weights stored as a state_dict)          |
| Loss function      | Binary cross-entropy (BCELoss) on sigmoid outputs                 |
| Optimiser          | Adam, learning rate 0.001                                         |
| Batch size         | 100                                                               |
| Maximum epochs     | 100                                                               |
| Early stopping     | Patience of 10 epochs on validation loss; best weights restored   |
| Regularisation     | Dropout (p = 0.5) before the output layer                         |
| Decision threshold | 0.5 during evaluation; adjustable (0.1 to 0.9) in the application |

Table 6. Training hyper-parameters.

### 5.3 Training behaviour

Training stopped automatically at epoch 31, when the validation loss had not improved for ten consecutive epochs; the lowest validation loss (about 0.19) occurred around epoch 21 and those weights were restored. Both curves in Figure 9 improve quickly during the first epochs and then flatten. Validation accuracy sits above training accuracy throughout; this is expected because dropout is active only during training, and it also indicates that the model is not over-fitting.

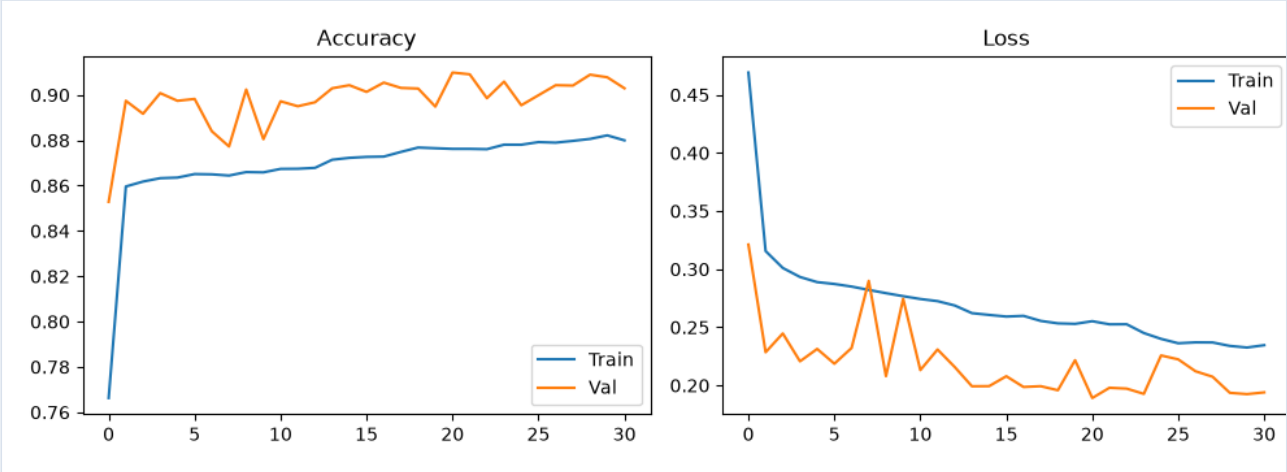


Figure 9. Training and validation accuracy (left) and loss (right) per epoch. Source: output of the training notebook.

| Epoch | Train loss | Train accuracy | Validation loss | Validation accuracy |
|-------|------------|----------------|-----------------|---------------------|
| 10    | 0.2767     | 0.8658         | 0.2745          | 0.8805              |
| 20    | 0.2529     | 0.8765         | 0.2216          | 0.8949              |
| 30    | 0.2326     | 0.8822         | 0.1925          | 0.9078              |

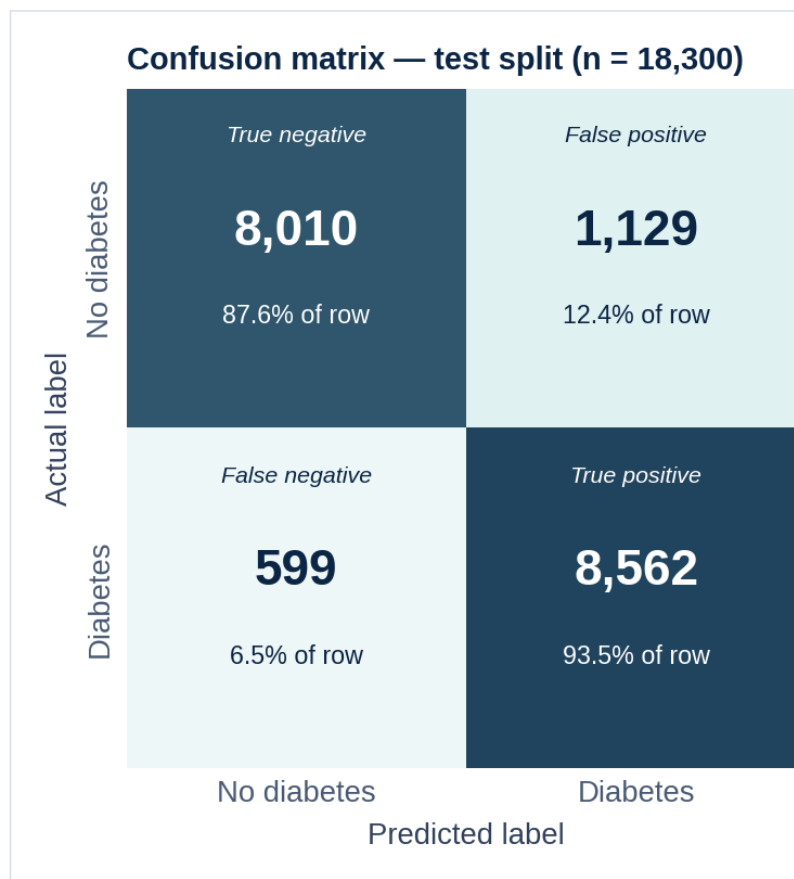
Table 7. Progress logged every 10 epochs.

## 6. Evaluation and Results

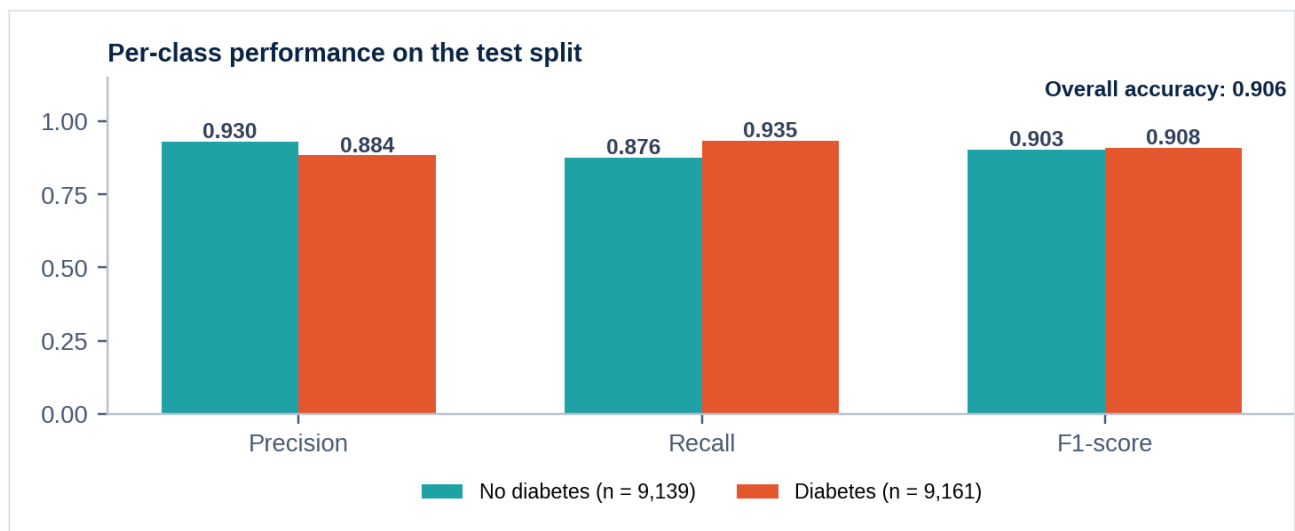
The final model was evaluated once on the held-out test split of 18,300 samples using a decision threshold of 0.5.

| Metric                    | Score  | Definition                                                      |
|---------------------------|--------|-----------------------------------------------------------------|
| Accuracy                  | 0.9056 | (TP + TN) / all samples                                         |
| Precision                 | 0.8835 | TP / (TP + FP): how many flagged patients are truly diabetic    |
| Recall (sensitivity)      | 0.9346 | TP / (TP + FN): how many diabetic patients are found            |
| F1-score                  | 0.9083 | Harmonic mean of precision and recall                           |
| Specificity               | 0.8765 | TN / (TN + FP): how many healthy patients are correctly cleared |
| Negative predictive value | 0.9304 | TN / (TN + FN)                                                  |
| Matthews correlation      | 0.8125 | Balanced single-number summary, from -1 to +1                   |

Table 8. Test-set metrics. The first four values are the notebook's output; the remaining three are derived from its confusion matrix.



**Figure 10.** Confusion matrix on the test split (rows are the actual class, columns the predicted class).



**Figure 11.** Precision, recall and F1-score for each class. Both classes are within a few points of each other.

## 6.1 Interpretation

- **High sensitivity.** The model finds 93.5% of diabetic patients and misses 599 of 9,161, which is the desirable direction for a screening tool where a missed case is costlier than a false alarm.
- **Moderate specificity.** 1,129 of 9,139 healthy patients (12.4%) are flagged as diabetic. Raising the decision threshold in the application trades some sensitivity for fewer false alarms.
- **Balanced behaviour.** Precision and recall are similar for both classes, so the network is not simply favouring one outcome.

- **Context is essential.** These numbers are measured on a class-balanced split that contains synthetic samples; Section 7 explains why they should not be read as real-world performance.

## 7. Critical Analysis and Supplementary Results

This section goes beyond the notebook. It examines how the reported metrics would change at the dataset's natural class balance and compares the network with simple reference models. All figures in this section were computed for this report; they are not part of the original notebook.

### 7.1 What the balanced test set hides

SMOTE only creates *positive* samples, so the negatives in the test split are real patients and the false-positive rate of 12.4% is a fair estimate. At the natural prevalence of 8.5%, that rate implies roughly 11,304 false alarms for every 7,944 correctly detected diabetic patients per 100,000 people (Table 9). The projected precision falls to about 41%, and overall accuracy (88.1%) would be below the 91.5% obtained by simply predicting “no diabetes” for everybody. The recall figure itself is likely optimistic too, because synthetic positives in the test split are interpolations of real positives that also appear in training.

| Quantity (per 100,000 people) | Balanced test (as reported) | Projected at 8.5% prevalence |
|-------------------------------|-----------------------------|------------------------------|
| True positives                | 8,562                       | 7,944                        |
| False negatives               | 599                         | 556                          |
| False positives               | 1,129                       | 11,304                       |
| True negatives                | 8,010                       | 80,196                       |
| Precision                     | 88.4%                       | 41.3%                        |
| Accuracy                      | 90.6%                       | 88.1%                        |

**Table 9.** Illustrative projection: the test-set true- and false-positive rates applied to the original 8,500 / 91,500 class split (assumes the recall stays at its reported, probably optimistic, value).

### 7.2 Reference baselines at the natural class balance

To put the network's behaviour in perspective, three simple references were evaluated on the original, unbalanced data.

| Model                                                        | Accuracy | Precision | Recall | F1    | Note                      |
|--------------------------------------------------------------|----------|-----------|--------|-------|---------------------------|
| Always predict “no diabetes”                                 | 91.5%    | -         | 0.0%   | -     | Majority-class baseline   |
| Two-condition rule (HbA1c ≥ 6.8 or glucose ≥ 220)            | 97.2%    | 100.0%    | 66.9%  | 0.802 | Descriptive, full dataset |
| Gradient Boosting (default settings, stratified 80/20 split) | 97.2%    | 98.8%     | 67.6%  | 0.803 | ROC-AUC 0.980             |

**Table 10.** Reference results on the natural class distribution (no oversampling). They are not directly comparable to the balanced-test numbers in Section 6.

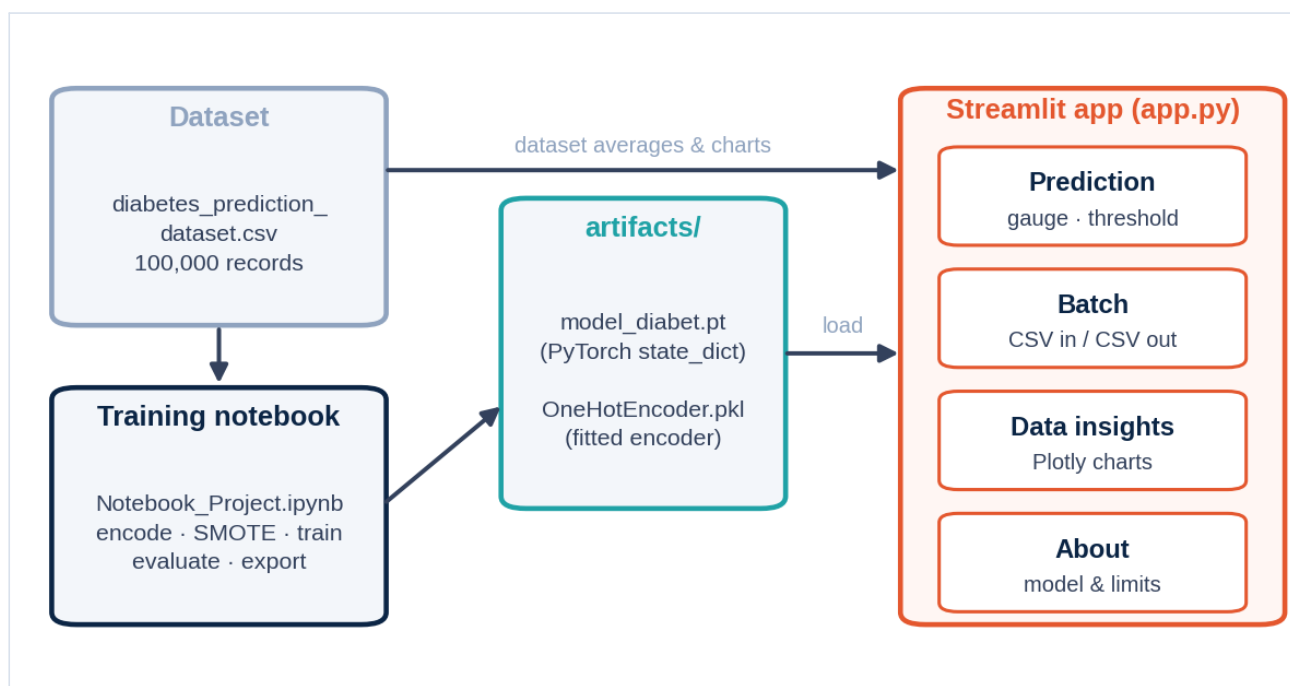
A tree-based model with default settings and no resampling reaches about 97% accuracy and a ROC-AUC of 0.98 on the natural distribution, which shows that the signal in the data is strong and that a well-configured network should be able to do at least as well. It also confirms that the hard part of this dataset is the ambiguous zone described in Section 3.5, where recall for the diabetic class stays limited for every model tried here.

## 7.3 Additional observations

- **SMOTE on one-hot features.** SMOTE interpolates every column, including the one-hot encoded ones, so synthetic rows contain fractional category values (for example 0.31 and 0.69 for two smoking categories). Such values never occur at inference time. A variant designed for mixed data (SMOTENC) or class weights would avoid this.
- **Uncalibrated probabilities.** Because the training classes were balanced, the output probability does not reflect the true 8.5% prevalence. The gauge in the application should be read as a relative risk score.
- **Single split.** All conclusions rest on one random split without a fixed seed and without cross-validation, so run-to-run variation is unknown.

## 8. The Streamlit Application

The trained artifacts are served by `app.py`, a single-file Streamlit application with a wide layout. Model and encoder are loaded once through `st.cache_resource`, and the dataset is cached with `st.cache_data`, so interaction stays responsive.



**Figure 12.** System architecture: the notebook produces two artifacts that the Streamlit application loads at start-up.

### 8.1 Interface

| Tab                  | What the user can do                                                                                                                                                                                                                                                                         |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prediction</b>    | Set patient details in the sidebar (gender, age, BMI, HbA1c, glucose, smoking history, hypertension, heart disease) and read the result on a gauge chart with a green or red risk banner. The decision threshold is adjustable; the patient's values are compared with the dataset averages. |
| <b>Batch</b>         | Upload a CSV, receive a diabetes probability and a "Diabetic / Not diabetic" label for each row, and download the scored file as predictions.csv.                                                                                                                                            |
| <b>Data insights</b> | Inspect record count, prevalence and feature count; view the class-distribution donut and correlation heatmap; pick a feature and see its class-wise histogram with the current patient marked.                                                                                              |
| <b>About</b>         | Model summary, preprocessing description and stated limitations.                                                                                                                                                                                                                             |

**Table 11.** The four tabs of the application.

## 8.2 Inference pipeline

For every prediction the application (1) collects the eight inputs into a one-row table, (2) one-hot encodes gender and smoking history with the saved encoder, (3) concatenates the six numeric columns with the nine encoded columns in the same order as during training, (4) feeds the 15-dimensional vector to the network in evaluation mode without gradient tracking, and (5) compares the returned probability with the chosen threshold.

| Input                        | Control    | Allowed range / options                            | Default |
|------------------------------|------------|----------------------------------------------------|---------|
| Gender                       | Select box | Female, Male, Other                                | Female  |
| Age                          | Slider     | 1 to 80 years                                      | 40      |
| BMI                          | Slider     | 10.0 to 70.0 (step 0.1)                            | 27.0    |
| HbA1c level                  | Slider     | 3.5 to 9.0 % (step 0.1)                            | 5.7     |
| Blood glucose                | Slider     | 80 to 300 mg/dL                                    | 120     |
| Smoking history              | Select box | never, No Info, current, former, ever, not current | never   |
| Hypertension / heart disease | Toggles    | Off / on                                           | Off     |
| Decision threshold           | Slider     | 0.10 to 0.90 (step 0.05)                           | 0.50    |

**Table 12.** Input controls of the Prediction tab. The ranges keep inputs within the interval seen during training.

## 8.3 Batch file format

The uploaded CSV must contain the eight feature columns below (any additional columns are preserved in the output). Unknown categories or missing columns are caught and reported as an on-screen error message.

```
gender,smoking_history,age,hypertension,heart_disease,bmi,HbA1c_level,blood_glucose_level
Female,never,54,0,0,27.3,6.6,140
Male,current,67,1,1,31.8,7.2,210
```

## 8.4 Running the project

```
git clone https://github.com/<your-username>/Diabetes-Predict.git
cd Diabetes-Predict
python -m venv venv && source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py # opens http://localhost:8501
```

To reproduce the training, install the extra notebook dependencies (`pip install imbalanced-learn matplotlib jupyter`) and open `Notebook_Project.ipynb`. The application requires Streamlit 1.35 or newer, PyTorch, scikit-learn, pandas, NumPy and Plotly.

## 9. Limitations and Future Work

### 9.1 Limitations

| Area          | Limitation                                                                                                | Consequence                                                                    |
|---------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Evaluation    | SMOTE applied before the split; balanced test set; single unseeded split                                  | Metrics are optimistic and not comparable to real-world prevalence             |
| Preprocessing | Scaler fitted but not applied; SMOTE interpolates one-hot columns                                         | Slower convergence; synthetic rows unlike real inputs                          |
| Calibration   | Probabilities learned on balanced classes                                                                 | Output is a relative score, not a true risk                                    |
| Data          | Discretised values, 3.85% duplicates, imputed-looking BMI, rule-like class boundaries, 18 "Other" genders | Limited generalisation to real patients; weak evidence for small subgroups     |
| Coverage      | Ages 0-80, glucose 80-300 mg/dL, eight features only                                                      | No extrapolation; no family history, medication, waist size, or lifestyle data |
| Validation    | No cross-validation, no external dataset, no clinical review                                              | Unknown stability and clinical validity                                        |

**Table 13.** Summary of known limitations.

#### Ethical note

The tool is intended for education and research. It must not be used to diagnose, to rule out diabetes, or to guide treatment. Predictions about individuals should always be confirmed with laboratory testing by qualified healthcare professionals.

### 9.2 Roadmap

| Priority   | Improvement                                                                                                                                        | Expected benefit                            |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1 - High   | Split first, then apply SMOTE (or class weights) to the training set only; evaluate on the untouched, imbalanced test set with a fixed random seed | Trustworthy, reproducible metrics           |
| 1 - High   | Apply a StandardScaler inside the pipeline and save it with the other artifacts                                                                    | Faster, more stable training                |
| 1 - High   | Report ROC-AUC, PR-AUC, calibration curve and threshold analysis                                                                                   | Metrics that are meaningful under imbalance |
| 2 - Medium | Benchmark against logistic regression, XGBoost and LightGBM; add stratified k-fold cross-validation                                                | Evidence that the network adds value        |
| 2 - Medium | Calibrate probabilities (Platt or isotonic); choose the threshold from a cost-sensitive analysis                                                   | Interpretable risk percentages              |
| 2 - Medium | Add SHAP explanations and per-patient feature contributions to the application                                                                     | Transparency for users                      |
| 3 - Later  | Pin dependency versions, add unit tests, Docker image and continuous integration; publish on Streamlit Community Cloud                             | Reproducible deployment                     |
| 3 - Later  | Validate on an independent, real-world dataset; write a model card                                                                                 | Assessment of external validity             |

**Table 14.** Prioritised improvement plan.

# 10. Conclusion

The project demonstrates a complete machine learning workflow: a 100,000-record dataset was explored and cleaned of obvious issues, encoded, balanced, and used to train a compact PyTorch network that reaches 90.6% accuracy and 93.5% recall on a balanced hold-out split. The model and its encoder are packaged as artifacts and exposed through a Streamlit application that supports single predictions, batch scoring and data exploration.

The analysis also shows where the work can grow. The strongest predictors, HbA1c and blood glucose, behave almost like deterministic thresholds in this dataset, so the evaluation protocol must focus on the ambiguous middle zone and on realistic class proportions. Following the roadmap in Section 9, chiefly a leakage-free split, feature scaling, calibrated probabilities and stronger baselines, would turn a solid learning project into a rigorously validated one.

# Appendix A. Model Input Vector

| Index | Feature                     | Source                    | Type       |
|-------|-----------------------------|---------------------------|------------|
| 0     | age                         | Numeric column            | Continuous |
| 1     | hypertension                | Binary column             | 0 / 1      |
| 2     | heart_disease               | Binary column             | 0 / 1      |
| 3     | bmi                         | Numeric column            | Continuous |
| 4     | HbA1c_level                 | Numeric column            | Continuous |
| 5     | blood_glucose_level         | Numeric column            | Continuous |
| 6     | gender_Female               | One-hot (gender)          | 0 / 1      |
| 7     | gender_Male                 | One-hot (gender)          | 0 / 1      |
| 8     | gender_Other                | One-hot (gender)          | 0 / 1      |
| 9     | smoking_history_No Info     | One-hot (smoking_history) | 0 / 1      |
| 10    | smoking_history_current     | One-hot (smoking_history) | 0 / 1      |
| 11    | smoking_history_ever        | One-hot (smoking_history) | 0 / 1      |
| 12    | smoking_history_former      | One-hot (smoking_history) | 0 / 1      |
| 13    | smoking_history_never       | One-hot (smoking_history) | 0 / 1      |
| 14    | smoking_history_not current | One-hot (smoking_history) | 0 / 1      |

**Table 15.** The 15-dimensional input vector, in the order expected by the network.

## Appendix B. Project Structure

```
Diabetes-Predict/  
|-- Dataset/  
|   |-- diabetes_prediction_dataset.csv    # raw data, 100,000 rows  
|-- artifacts/  
|   |-- model_diabet.pt                  # trained PyTorch weights  
|   |-- OneHotEncoder.pkl                # fitted encoder  
|-- Notebook_Project.ipynb              # EDA, training, evaluation  
|-- app.py                             # Streamlit application  
|-- requirements.txt                    # streamlit, torch, scikit-learn, pandas, numpy, plotly  
|-- LICENSE                             # MIT  
|-- README.md
```

## References

- Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- Kingma, D. P., and Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929-1958.
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232.
- Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.